



Good — the accessibility framing actually makes the app choice much easier, because it gives you a clear "who is this for" and a video editor is the textbook feature-overload case.

App recommendation: OpenCut

OpenCut is an open-source CapCut alternative, MIT-licensed, and built as a Next.js web app with the UI in `src/components/`, state in Zustand stores, and TypeScript types throughout. Why it's the right pick:

- **A video editor is the canonical feature-subsetting demo.** "I just want to trim this clip" collapsing a timeline, effects panel, properties inspector, media library, and export settings down to two controls is instantly legible to a judge from across the room. It's the example you gave me yourself.
- Zustand + component-per-panel structure means your region registry is mostly a matter of tagging existing panels, not refactoring.
- Docker/Postgres/Redis are only needed for backend services — you can skip them if you're only running the frontend, which is all you need.

One trap: there are two versions. The rewrite is at `new.opencut.app` and the maintainers explicitly say they aren't taking outside contributions while the architecture is in flux; the previous version at `opencut-app/opencut-classic` is the one to reach for today. Use classic. Don't let an agent auto-clone the wrong one at hour 0.

Alternative if OpenCut fights you in hour 1: Excalidraw. Smaller, cleaner, very recognizable — but it already ships i18n, which weakens your translation pillar, and it has less UI to strip, which weakens your hero pillar. Try OpenCut first, hard-timebox the decision to 45 minutes.

Hardware — you're right, I cut it too fast

Here's the version I think is both buildable in 26h and *conceptually* the strongest, rather than just a Pi in a box:

Four physical buttons whose functions are reassigned by the AI, with their labels rendered on-screen right next to them. Same four buttons, completely different tool depending on what you said you're doing. Say "I'm trimming a clip" and they become `mark in / mark out / preview / export`. Say "I'm adding captions" and they become `add / prev / next / style`. That's the hardware claim made physical in one glance — and it's the thing a software-only demo literally cannot do.

Why this is safe to build: the buttons are GPIO → websocket → your existing mutation pipeline. No new model work, no new schema, it reuses everything from the software path.

Build spec:

- Raspberry Pi 5 + small DSI touchscreen (4–5"), Chromium in kiosk mode.
- 4 tactile buttons on GPIO, one push-to-talk button, a USB mic.
- 3D-printed handheld shell — print time is passive, and this is squarely your wheelhouse. Model it early so it's printing while you code.
- **The Pi renders only the subsetting UI, never the full editor.** This is important both for performance (a Pi will choke on the full editor's video preview) and for the narrative — the device *physically can't* show you the full app, which is the point. Laptop shows the full desktop app; Pi shows what the AI decided you actually need.

Baseten — two uses, ranked

1. Host a translation model. Do this one first; no training required. Deploy NLLB-200 or M2M100 as a Baseten endpoint and batch-translate your tagged UI strings. This is the single highest-value Baseten move because it removes the only slow call in your entire pipeline — pillar 3 goes from "one GPT call, ~2–4s" to sub-second, and your speed story stays intact across all three pillars instead of breaking on the last one. Steps:

- Deploy NLLB-200 (distilled 600M) via Baseten's model library. *Done when:* a batch of 40 UI strings round-trips in under 600ms.

- Wire Jev's extracted target-language field directly to the endpoint. *Done when:* "switch to French" → fully translated UI with no LLM in the path.
- Pre-warm the endpoint before the demo. *Done when:* first call after 10 min idle is still under 1s.

2. Fine-tune a small model for novel design tokens. Stretch goal only. Jev handles your enumerated token space, but it can't invent a hex code. A fine-tune fills that gap:

- Synthetically generate ~3–5k pairs of (aesthetic description → design-token JSON) using Gemini/GPT offline. *Done when:* 100 hand-spot-checked samples are valid and varied.
- Fine-tune a 1–3B model (Qwen, Llama) on Baseten. *Done when:* held-out descriptions produce schema-valid token sets ≥95% of the time.
- Deploy behind a "surprise me" path. *Done when:* novel-palette generation completes in under 500ms.

Be honest with yourselves about this one: a fine-tune inside 26 hours is a real risk of eating 6 hours for a marginal upgrade over Jev's curated palettes. I'd gate it — only start it if you're ahead of schedule at hour 16.

Revised 26h plan

Phase 0 — Foundation (all, hrs 0–3)

1. Clone and run OpenCut classic, frontend only. *Done when:* it runs and is recognizable. Hard stop at 45 min; fall back to Excalidraw.
2. (A) Region registry — tag every panel/toolbar with a stable ID. *Done when:* 12–16 regions enumerate correctly.
3. (A) Route styling through CSS custom properties. *Done when:* flipping variables in devtools fully reskins it.
4. (C) Partial-transcript streaming with stale-request cancellation. *Done when:* only the newest partial is ever in flight.
5. (B) One trivial Jev query end to end. *Done when:* typed output + confidences + latency logged. Benchmark direct TypeSafe vs Backboard here and hard-code the faster one.
6. (Massah) Start the shell print. *Done when:* it's on the bed.

Phase 1 — Feature-subsetting (hrs 3–10) 7. (B) Jev schema: transcript + region list → per-region visibility. *Done when:* 5 goal phrases produce sensible sets. 8. (B) Policy gates (`is_command` , `complete` , `destructive`) → ACT/WAIT/IGNORE. *Done when:* 20 utterances, zero false acts. 9. (A) Mutation applier. *Done when:* a hand-fed set visibly strips the editor, no reload. 10. (All) Voice → Jev → mutation, wired. *Done when:* "I just want to trim this clip" reshapes the editor in under 1.5s from last word, 3× running.

Phase 2 — Reskinning (hrs 10–13) 11. (A) Token space: palettes, fonts, density, radius, contrast mode. *Done when:* every combination renders without breakage. 12. (B) Jev selects across all axes in one call. *Done when:* "make it high contrast and huge text" and "make it pink and soft" both land correctly.

Phase 3 — Translation (hrs 13–16) 13. (A) Tag static copy with stable IDs. *Done when:* a hand-written map swaps everything cleanly. 14. (B) Baseten NLLB endpoint + Jev language extraction. *Done when:* full UI translates in under 1s.

Phase 4 — Hardware (hrs 13–19, parallel) 15. Pi + touchscreen + kiosk Chromium pointed at the subsetted view. *Done when:* it boots straight into the app unattended. 16. GPIO buttons → websocket → action dispatch. *Done when:* pressing button 2 fires the correct action on the running app. 17. Dynamic button labels rendered beside each physical button. *Done when:* changing stated intent visibly relabels and remaps all four. 18. Shell assembly. *Done when:* it's handheld and survives being passed to a judge.

Phase 5 — Persistence + telemetry (hrs 16–21) 19. (A) Profile store — customizations survive reload. *Done when:* reload restores the exact prior state. 20. (C) Telemetry panel: gates vs thresholds, intent distribution, word→decide→done timings, running cost. *Done when:* readable at projector distance, updates per partial. 21. (B) Whitelist validation before any mutation hits the DOM. *Done when:* 5 malformed payloads rejected.

Phase 6 — Rehearsal (hrs 21–25) 22. Full run: subset → reskin → language → hand the device to someone. *Done when:* 3 clean consecutive runs in your slot. 23. Pre-recorded fallback. *Done when:* it matches narration beat-for-beat.

Hr 25–26: buffer. No new features. Not even small ones.

On coding agents

They'll genuinely save you time on steps 2, 3, 13, 19, 20 — tagging, refactoring to tokens, and building the telemetry UI are exactly the mechanical, well-specified work agents are good at. They will *not* save you time on Jev schema tuning, threshold calibration, latency debugging, hardware, or rehearsal, and those are most of your actual risk.

The failure mode to watch: agents generate working-looking code fast enough that you arrive at hour 18 with four components nobody has read and no integration. Force step 10 (first full voice→UI loop) to happen on schedule even if the pieces are ugly, and have one person own integration rather than all three generating in parallel.

Two more questions

1. **Are you demoing on a projector or at a table?** It changes everything about the telemetry panel and whether the handheld device is even visible. If it's a judging table, the Pi becomes your primary surface and the laptop is secondary — that flips your build priority.
2. **Which accessibility persona is the demo actually narrated as?** ADHD and blind pull in opposite directions — ADHD wants aggressive visual subsetting, blind wants TTS and keyboard/button-driven navigation with the visual layer mattering less. Doing both dilutes. I'd pick ADHD/cognitive-load as the narrated persona since it showcases pillars 1 and 2 directly, and mention blind-user support as an extension you've architected for. But this is your call and it should be made before hour 3, since it determines what "correct" looks like in step 7.